

Responsive Design

Design for a small screen first, then add space and structure for larger screens without hiding essential information.



Learning outcomes

By the end of this lecture you can build and test a layout that works from 320px to 1920px.

Reason

Explain why mobile-first, min-width breakpoints produce more resilient CSS than desktop-first max-width overrides.

Build

Write fluid layouts with flexbox and grid, fluid images, and fluid type that adapt without a fixed breakpoint list.

Test

Verify a layout at multiple real breakpoints and confirm no essential content or navigation is hidden on small screens.

Why responsive design exists

One HTML document is now viewed on an unbounded range of screen sizes.

In 2010, Ethan Marcotte coined “responsive web design” to describe a single codebase that adapts its layout to the viewport, instead of maintaining separate mobile and desktop sites.

Before

After



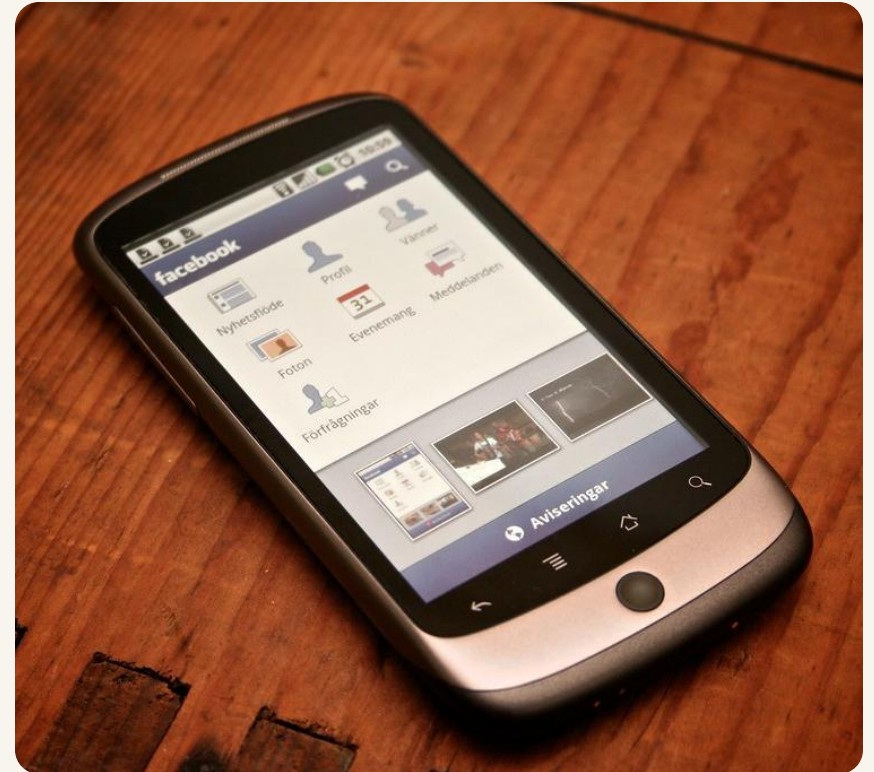
Mobile-first as a discipline

Design and write CSS for the most constrained viewport, then progressively add.

Mobile-first is not “make it work on phones too.” It means your default, unqualified CSS targets a narrow single-column viewport, and every media query adds capability for more space — never removes it.

Why it matters

Consequence



The viewport meta tag

Without it, mobile browsers render a fake 980px-wide desktop page and zoom it out.

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

width=device-width

Sets the layout viewport's width to the device's actual CSS pixel width, instead of an assumed 980px.

initial-scale=1

Sets the initial zoom so one CSS pixel maps to one viewport pixel at load — no automatic zoom-out.

Media queries: the syntax

A conditional block of CSS that applies only when a feature of the viewport matches.

```
/* mobile-first: base styles apply everywhere */
.card-grid { display: grid; grid-template-columns: 1fr; gap: 1rem; }

@media (min-width: 48rem) {
  .card-grid { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 64rem) {
  .card-grid { grid-template-columns: repeat(3, 1fr); }
}
```

min-width vs max-width

Mobile-first uses min-width. Desktop-first (retrofitting an old site) uses max-width.

min-width (mobile-first)

@media (min-width: 48rem) { ... } adds styles as the viewport grows.
Base styles are the narrow case; nothing needs undoing.

max-width (desktop-first)

@media (max-width: 48rem) { ... } overrides desktop styles for narrow viewports — every override fights the cascade against base styles meant for wide screens.

This course uses min-width breakpoints throughout: write for the smallest viewport first, then layer on capability.

Choosing breakpoints from content, not devices

Pick a breakpoint where YOUR layout breaks, not a device's marketing width.

Device widths change constantly — there is no fixed list of “phone / tablet / desktop” pixel widths to target. Instead, resize your own browser window until the content itself starts to look cramped or the line length gets awkward, and set the breakpoint there.

~30rem (480px)

small phones in landscape, or a stacked card grows a second column

~48rem (768px)

common tablet/narrow-window breakpoint for many card and nav layouts

~64rem (1024px)

a comfortable point to reveal a persistent sidebar or 3+ column grid

~80rem (1280px)

constrain max content width so lines don't become unreadably long

Fluid layout, not fixed widths

Percentages, max-width and relative units let a layout resize instead of clipping or leaving gaps.

```
.page {  
  width: 100%;  
  max-width: 60rem; /* caps line length on wide screens */  
  margin-inline: auto; /* centers within the viewport */  
  padding-inline: 1rem; /* keeps a gutter at any width */  
}
```

A fixed width:1024px container overflows a 375px phone screen and forces horizontal scrolling. width:100% with a max-width cap fills small screens and stays readable on large ones.

Flexible images

An image must shrink with its container or it breaks the layout, not the other way around.

```
img, video, svg {  
  max-width: 100%;  
  height: auto;  
  display: block;  
}
```

Why height:auto matters

max-width:100% alone with a fixed height attribute distorts the aspect ratio. height:auto preserves it as width scales down.

This is a baseline reset most CSS resets include — without it, a single oversized image can force horizontal scrolling on the whole page.

Responsive images: srcset and sizes

Serve a smaller file to a small viewport instead of shrinking a huge one with CSS.

```

```

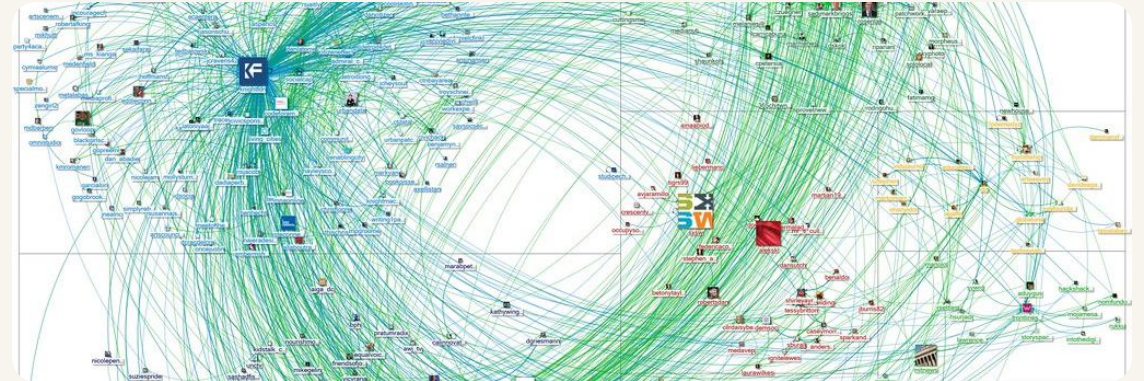
What this buys you

The browser picks the best-matching file for the viewport and pixel density, so a phone downloads a 400px image instead of the same 1200px file a desktop gets — real bandwidth and load-time savings, not just a CSS resize.

Flexbox for responsive layout

flex-wrap and order let items reflow without a media query for every case.

```
.nav-list {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 0.5rem 1rem;  
}  
.nav-list a { flex: 1 1 8rem; }
```



How this behaves

flex-wrap:wrap lets items drop to a new row when space runs out, and flex:1 1 8rem gives each item a flexible basis so items grow to fill remaining space evenly — no explicit breakpoint needed for this behaviour.

CSS Grid: auto-fit and minmax

A single declaration that reflows column count based on available width.

```
.card-grid {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(16rem, 1fr));  
  gap: 1.5rem;  
}
```

Reading this line

auto-fit creates as many 1fr columns as fit; minmax(16rem, 1fr) sets each column's minimum useful width to 16rem and lets it grow to share leftover space evenly.

This single rule replaces three separate media-query breakpoints from the earlier card-grid example — the grid recalculates itself continuously as the viewport resizes.

Container queries: beyond the viewport

A component that responds to the size of its own container, not the whole page.

```
.sidebar { container-type: inline-size; }

@container (min-width: 20rem) {
  .card { grid-template-columns: 6rem 1fr; }
}
```

Why this matters

A media query only knows the viewport width. A card component placed in a wide main column needs different styling than the same component placed in a narrow sidebar — container queries let the component decide for itself, based on the space it is actually given.

Fluid typography with clamp()

Type size scales continuously with viewport width, between a floor and a ceiling.

```
h1 {  
  font-size: clamp(1.75rem, 4vw + 1rem, 3rem);  
}
```

Reading clamp()

clamp(MIN, PREFERRED, MAX): never smaller than 1.75rem, never larger than 3rem, and in between grows fluidly with 4vw + 1rem.

Why not just vw?

font-size:4vw alone has no floor or ceiling — text can become unreadably small on a phone or absurdly large on an ultrawide monitor.

Navigation for small screens

The hamburger icon compresses navigation — but compression is not the same as hiding.

A hamburger button behind an icon-only trigger can save space, but every item inside it must still be reachable, labelled and keyboard-operable — collapsing navigation must never mean removing it.

Minimum requirements

Common failure



“Don't hide essential information”

This week's brief names the core risk of small-screen design directly.

Compressing a layout for a small screen is not the same as deciding what to cut. A price, a phone number, a safety warning, or the primary call-to-action must survive the transition to 320px — only the presentation may change.

The priority+ pattern

Show the most important nav items directly; group the rest behind a labelled “More” control — nothing disappears, low-priority items are just one tap further away.

Progressive disclosure vs deletion

Collapsing a long description behind a “Read more” toggle is progressive disclosure. Removing that description from the mobile build entirely is deletion — only the first is acceptable.

Touch targets

A finger is not a mouse pointer — interactive elements need real physical size.

WCAG 2.5.5 (Target Size) recommends touch targets of at least 44 × 44 CSS pixels, with adequate spacing between adjacent targets so an imprecise tap doesn't hit the wrong one.

In practice

Placeholder text for the In practice section.

Spacing

Placeholder text for the Spacing section.



Testing responsively with DevTools

Resize a real browser, don't just imagine the breakpoints.

1

Open the device toolbar (Ctrl/Cmd+Shift+M) to simulate common viewport widths and touch input.

2

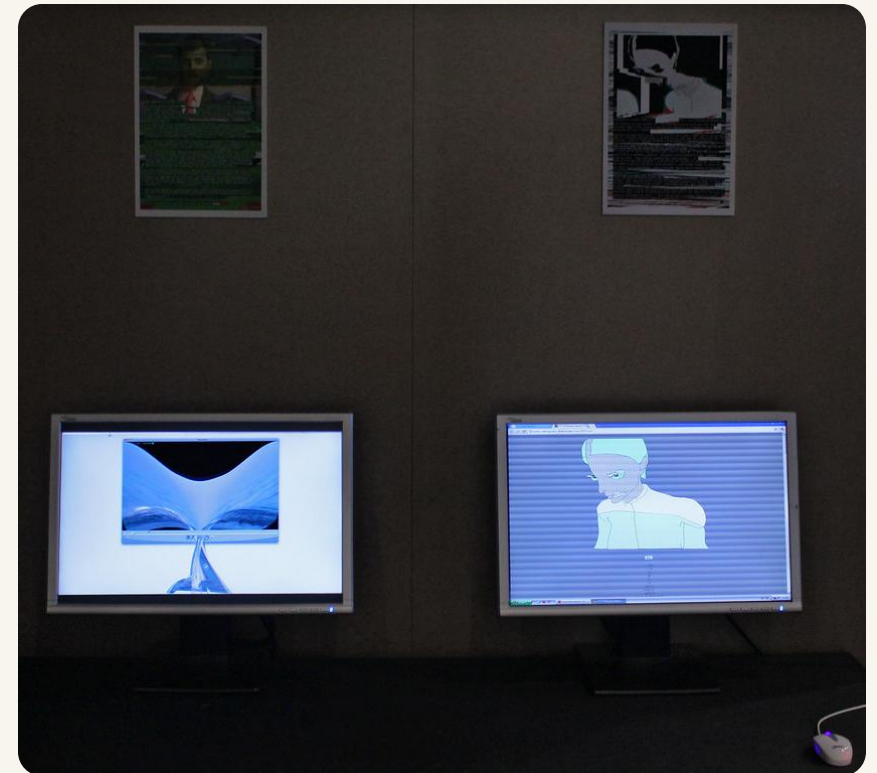
Drag the responsive viewport handle continuously through 320–1920px to find where content breaks, not just at preset device widths.

3

Check the network throttling panel — mobile users are often on slower, higher-latency connections than your desktop dev machine.

4

Repeat every check on at least one real physical phone or tablet: emulation does not reproduce real touch behaviour or font rendering.



Testing on real devices and tablets

Emulation approximates; a tablet in portrait and landscape is a distinct real case.

A tablet is not “a small desktop” or “a big phone” — it is its own case, and it commonly rotates between portrait and landscape mid-session, changing its effective breakpoint without a page reload.

What to check

How to check: Use a tablet in portrait and landscape, and check the layout and content.

Common breakpoint

How to check: Use a tablet in portrait and landscape, and check the layout and content.



Common responsive design mistakes

Four failures this course watches for in project review.

Fixed pixel widths

A hard-coded width:960px container overflows any viewport narrower than 960px, forcing horizontal scroll.

Content hidden with display:none on mobile

Hiding a whole section at a breakpoint (rather than restructuring it) usually means real content was deleted, not adapted.

Tiny, cramped touch targets

Icon buttons styled only for a mouse cursor become unusable once a real thumb has to tap them.

Untested orientation change

A layout that only looks right in portrait can break completely the moment a tablet or phone rotates to landscape.

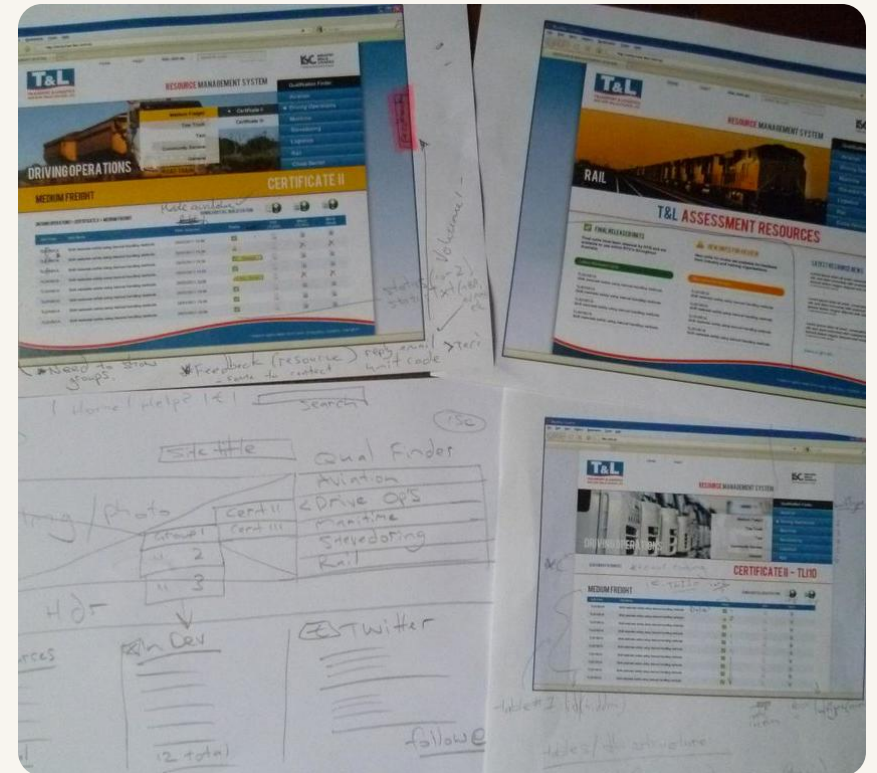
Case study, before: a fixed desktop layout

A real transformation, step one — what the untouched design looked like.

The original design used a fixed 1024px three-column layout: a 240px sidebar, a 600px content column, and a 184px widget rail — all in absolute pixels, with no media queries at all.

What broke

Root cause



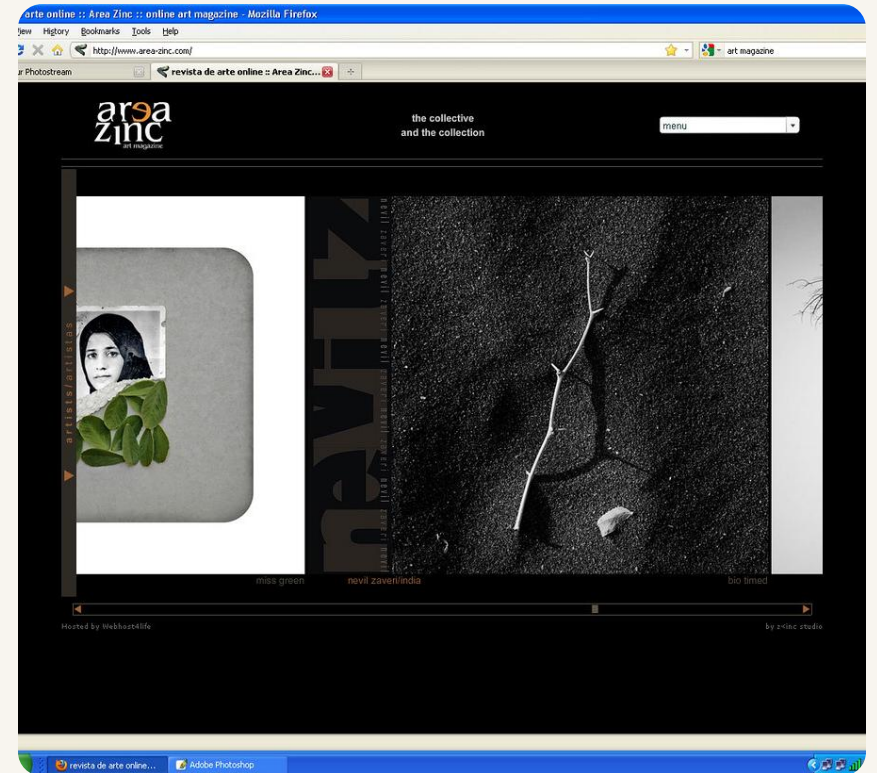
Case study, after: mobile-first rebuild

The same content, restructured with the techniques from this lecture.

Base styles now stack sidebar, content and widget rail in one column with fluid widths. A single `min-width:64rem` media query switches to the three-column grid using CSS Grid's `repeat()/minmax()`.

Result

Measured outcome



Breakpoint testing checklist

Four widths to check on every layout before calling it done.

320px	smallest common phone width — no horizontal scroll, all text and controls reachable
-------	---

768px	small tablet / narrow window — check the first added breakpoint's transition
-------	--

1024px	tablet landscape / small laptop — check any second breakpoint and nav changes
--------	---

1440px+	large desktop — check max-width caps keep line length readable, no awkward empty gutters
---------	--

Studio task

Rebuild a fixed layout as a mobile-first responsive page.

Brief

Take a page you built in an earlier week (or a simple 3-section page) and restyle it mobile-first: base styles for a single column, then add min-width breakpoints at ~48rem and ~64rem.

Constraints

No fixed pixel widths on any container. All images use max-width:100%. Every interactive control keeps a real 44px touch target at 320px width.

Evidence to submit

Screenshots at 320px, 768px and 1024px, plus 100 words on the one piece of content or navigation you were careful not to hide on the smallest screen.

Review checklist

How responsive work is assessed in project review.

Mobile-first CSS

Base styles target the narrowest viewport; media queries use min-width to add, not override.

No hidden essentials

Every critical piece of content or navigation remains reachable at 320px, even if repositioned.

Fluid images and type

Images use max-width:100%/height:auto; type does not become unreadably small or huge at any width.

Touch-target size

Interactive elements meet at least 44×44px with adequate spacing on touch viewports.

Connections to earlier weeks

Responsive design builds directly on semantic structure and CSS layout already covered.

Week 2 — semantic HTML

A correct landmark and heading structure means reflow at any breakpoint keeps meaning intact — responsive CSS cannot fix broken markup.

Week 4 — layout with CSS

Flexbox and Grid from Week 4 are the same tools used here; this week adds the media-query and container-query layer on top.

Week 8 (coming) — accessibility review

Touch-target size and keyboard-operable navigation from this week are directly re-checked in the Week 8 audit.

Assignment 3

This week's studio task is direct preparation for the interaction-and-forms assignment: a form must also stay usable at 320px.

Further practice and resources

Before next session.

Responsive images reference

MDN, Responsive images: https://developer.mozilla.org/en-US/docs/Web/HTML/Responsive_images

Container queries reference

MDN, CSS containment / container queries: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_containment/Container_queries

Touch target guidance

W3C WAI, Understanding Success Criterion 2.5.5 Target Size: <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>

Bring to Week 7

A screenshot of one form you use often (login, checkout, search) to critique its labels and error handling.

Next week

Week 7: Interaction and forms.

Build clear navigation and an accessible form. Test focus order, labels, errors and keyboard access — the interaction layer that sits on top of the responsive structure built this week.